



دانشکده سامانه‌های هوشمند

بینایی ماشین

استاد درس: دکتر حائیه نادری

پیاده سازی و مقایسه ی فیلتر های فرکانسی در حوزه ی Fourier

IMPORTS

```
import cv2
```

از این کتابخانه برای خواندن، نوشتن و انجام عملیات پردازش تصویر استفاده می شود.

```
import numpy as np
```

این کتابخانه، کتابخانه اصلی برای محاسبات عددی، کار با آرایه ها و اجرای محاسبات تبدیل فوری (FFT) است

```
import matplotlib.pyplot as plt
```

در این تمرین از این کتابخانه برای نمایش تصاویر و ذخیره سازی خروجی های بصری آزمایش به کار می رود.

```
import pandas as pd
```

برای سازماندهی نتایج تست در قالب DataFrame و ذخیره آنها در فایل CSV استفاده می شود.

PREPARE DATA

Read grayscale image

```
img = cv2.imread("1.png", 0)
```

برای لود کردن تصویر 1.png از تابع imread از کتابخانه Opencv استفاده میکنیم. * یعنی یعنی تصویر به صورت Grayscale خوانده شود. در نهایت، تصویر در متغیر img ذخیره شده است.

Display original image

```
plt.figure(figsize=(6,6))
plt.imshow(img, cmap='gray')
plt.title("Original Image")
plt.axis("off")
plt.show()
```

این قطعه کد با استفاده از کتابخانه Matplotlib تصویر خوانده شده را نمایش می دهد.

- ابتدا با `plt.figure(figsize=(6,6))` یک شکل با ابعاد ۶×۶ اینچ ایجاد می شود.
- سپس با `plt.imshow(img, cmap='gray')` تصویر در حالت خاکستری نمایش داده می شود.
- با `plt.title()` عنوانی برای تصویر تعیین می گردد.
- دستور `plt.axis("off")` محورها را مخفی می کند.
- در نهایت `plt.show()` شکل نهایی را نمایش می دهد.

Compute 2D Fourier Transform

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) e^{-j2\pi \left(\frac{ux}{M} + \frac{vy}{N} \right)}$$

```
f = np.fft.fft2(img)
```

این دستور تبدیل فوری دوبعدی تصویر `img` را محاسبه می کند. نتیجه در متغیر `f` ذخیره می شود و نشان دهنده نمایش تصویر در حوزه فرکانس است. در این فضا، اطلاعات مربوط به شدت تغییرات و الگوهای فرکانسی تصویر قابل بررسی خواهد بود.

Computing and scaling Magnitude

$$\text{Magnitude} = |F(u, v)| = \sqrt{\text{Re}^2(u, v) + \text{Im}^2(u, v)}$$

$$\log(1 + |F(u, v)|)$$

```
magnitude_before = 20 * np.log(np.abs(f) + 1)
```

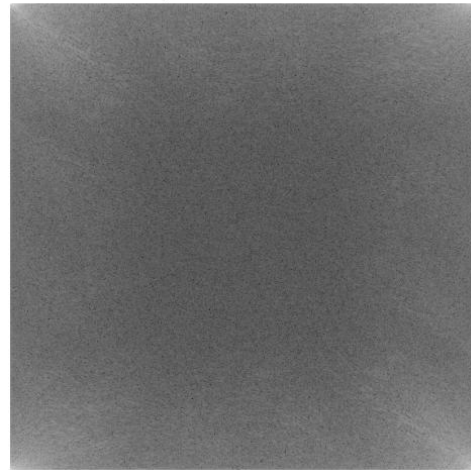
این قطعه کد، تصویر خام تبدیل فوری را به یک طیف دامنه (Magnitude Spectrum) تبدیل می کند که برای چشم انسان و نمایش های گرافیکی، مفهوم و قابل درک است.

- `np.log()`: دامنه فرکانس ها در تصاویر معمولاً دارای دامنه ی تغییرات بسیار وسیعی است. مقیاس لگاریتمی باعث می شود مقادیر کوچک (فرکانس های بالا) به نحوی فشرده و تقویت شوند که در کنار مقادیر بزرگ (فرکانس های پایین) در یک تصویر قابل مشاهده باشند.
- `np.abs(f)`: با این تابع، `magnitude` اعداد مختلط را محاسبه می کنیم.
- `+1`: این عدد برای پایداری عددی (Numerical Stability) اضافه شده است. از آنجایی که لگاریتم صفر تعریف نشده است با جمع کردن با 1، مطمئن می شویم که لگاریتم عدد صفر (که منطقاً کمترین مقدار است) به $\log(1) = 0$ تبدیل می شود و از خطای ریاضی جلوگیری می شود.
- `20`: این ضریب مقیاس دهی برای تبدیل واحدها به دسی بل (dB) است که استاندارد متداولی در پردازش سیگنال برای نمایش مقیاس های لگاریتمی محسوب می شود. اگر نتیجه در 20 ضرب نشود، در این حالت همچنان طیف فرکانسی نمایش داده می شود و از نظر شکل کلی تصویر تفاوت زیادی ایجاد نمی شود. اما مقادیر دیگر در مقیاس دسی بل (dB) نخواهند بود.

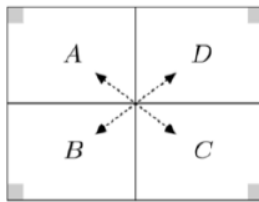
Spectrum before shifting



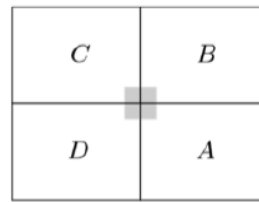
Spectrum Before Shift



Spectrum after shifting



(a)

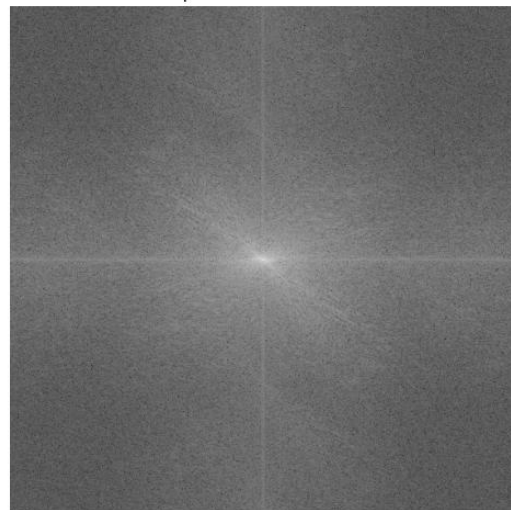


(b)

```
fshift = np.fft.fftshift(f)
magnitude_after = 20 * np.log(np.abs(fshift) + 1)
```

دستور `np.fft.fftshift` برای جابجایی مرکز طیف فرکانسی استفاده می‌شود. در خروجی پیش فرض `fft2`، مؤلفه‌ی فرکانس صفر (DC) در گوشه‌ی تصویر قرار دارد. تابع `fftshift` این مؤلفه را به مرکز ماتریس منتقل می‌کند. در نتیجه، طیف فرکانسی به شکل قابل فهم‌تری برای نمایش و تحلیل بصری آماده می‌شود. سپس مانند قبل، دامنه‌ی طیف جابه‌جاشده به مقیاس لگاریتمی تبدیل می‌شود تا قابل نمایش باشد، سپس با `matplotlib` نمایش داده و تصویر ذخیره می‌گردد.

Spectrum After Shift



Ideal Low Pass Filter

این تابع برای ساخت یک ماسک فیلتر پایین گذر ایده آل (Ideal Low-Pass Filter) طراحی شده است.

```
def ideal_lowpass(shape, D0):
    rows, cols = shape
```

ورودی این تابع، ابعاد طیف فرکانسی و اندازه D0 یا همان شعاع دایره فرکانس قطع است. ابعاد عکس در متغیرهای rows, cols ذخیره میشوند.

```
crow, ccol = rows // 2, cols // 2
```

این خط، مختصات مرکز ماتریس را محاسبه می کند تا بتواند فاصله پیکسل ها از مرکز را به دست آورد.

```
mask = np.zeros((rows, cols), np.uint8)
```

یک صفحه کاملاً سیاه با همان ابعاد تصویر ورودی ایجاد میشود که به صورت پیش فرض تمام پیکسل های آن ۰ (مسدود) هستند.

```
for u in range(rows):
    for v in range(cols):
```

حلقه اصلی ساخت ماسک: با استفاده از حلقه for تو در تو، درایه های ماتریس به نوبت انتخاب میشوند.

```
D = np.sqrt((u - crow)2 + (v - ccol)2)
```

$$D(u, v) = \sqrt{\left(u - \frac{P}{2}\right)^2 + \left(v - \frac{Q}{2}\right)^2}$$

crow و ccol همان P/2 و Q/2 هستند (مرکز طیف).

D، فاصله اقلیدسی هر پیکسل (u, v) را از مرکز محاسبه می کند.

```
if D <= D0:
    mask[u, v] = 1
```

همان تعریف فیلتر پایین گذر ایده آل است

$$H(u, v) = \begin{cases} 1 & D(u, v) \leq D_0 \\ 0 & D(u, v) > D_0 \end{cases}$$

```
return mask
```

خروجی ماتریسی است که در مرکز آن یک دایره سفید (۱) و در اطراف آن سیاهی (۰) قرار دارد

این تابع Butterworth LPF را پیاده‌سازی می‌کند، که نسخه‌ی نرم‌تر، واقعی‌تر و از نظر بینایی طبیعی‌تر فیلتر پایین‌گذر ایده‌آل است.

```
def butterworth_lowpass(shape, D0, n):
    rows, cols = shape
    crow, ccol = rows // 2, cols // 2
    mask = np.zeros((rows, cols), np.float32)
    for u in range(rows):
        for v in range(cols):
            D = np.sqrt((u - crow)2 + (v - ccol)2)
```

این تابع علاوه بر ورودی‌های تابع ایده‌آل، مرتبه n یا همان order را دریافت می‌کند.

```
mask[u, v] = 1 / (1 + (D / D0)(2 * n))
```

مقدار فیلتر مطابق با فرمول Butterworth LPF تعیین می‌شود:

$$H(u, v) = \frac{1}{1 + \left[\frac{D(u, v)}{D_0} \right]^{2n}}$$

```
return mask
```

خروجی یک ماتریس دوبعدی با مقادیر بین ۰ و یک است که مثل یک ماسک گذر پایین نرم است.

Ideal High Pass Filter

این تابع یک ماسک فیلتر بالاگذر ایده‌آل (Ideal High-Pass Filter) در حوزه فرکانس ایجاد می‌کند. هدف آن حذف فرکانس‌های پایین (نزدیک مرکز طیف) و عبور دادن فرکانس‌های بالا است.

```
def ideal_highpass(shape, D0):  
    rows, cols = shape  
    crow, ccol = rows // 2, cols // 2
```

```
mask = np.ones((rows, cols), np.uint8)
```

یک ماتریس به اندازه تصویر ساخته می‌شود که همه مقادیر آن ۱ هستند. این یعنی در ابتدا فرض می‌کنیم همه فرکانس‌ها عبور داده می‌شوند. (دقیقاً برعکس پایین گذر که در ابتدا همه مقادیر را قرار دادیم)

```
for u in range(rows):  
    for v in range(cols):  
        D = np.sqrt((u - crow)2 + (v - ccol)2)  
        if D <= D0:  
            mask[u, v] = 0
```

$$H(u, v) = \begin{cases} 0 & \text{if } D(u, v) \leq D_0 \\ 1 & \text{if } D(u, v) > D_0 \end{cases}$$

اگر فاصله از مرکز کمتر یا مساوی D_0 باشد:

- آن نقطه متعلق به فرکانس‌های پایین است.
- بنابراین مقدار ماسک ۰ می‌شود (حذف می‌شود).

در نتیجه:

- داخل دایره با شعاع D_0 — مقدار ۰ (حذف فرکانس‌های پایین)
- خارج از دایره — مقدار ۱ (عبور فرکانس‌های بالا)

```
return mask
```

خروجی ماتریسی است که در مرکز آن یک دایره سیاه (۰) و در اطراف آن سفید (۱) است.

Function for applying filters and displaying results

این تابع، مرحله‌ی نهایی پردازش تصویر در حوزه فرکانس را انجام می‌دهد. یعنی بعد از اینکه تصویر را به حوزه فرکانس بردیم، فیلتر را اعمال کردیم و حالا می‌خواهیم خروجی را ببینیم و ذخیره کنیم.

```
def apply_filter(filter_mask, filter_name):  
    filtered = fshift filter_mask
```

در این خط، ضرب نقطه‌ای (Element-wise) بین تبدیل فوریه تصویر (که قبلاً شیفت داده شده و در متغیر fshift ذخیره شده) و ماسک فیلتر انجام میشود.

در حوزه فرکانس، فیلترینگ با ضرب انجام می‌شود (فرکانس‌های ناخواسته حذف یا تضعیف می‌شوند)

```
f_ishift = np.fft.ifftshift(filtered)
```

چون قبلاً از fftshift برای انتقال فرکانس صفر به مرکز تصویر استفاده شده، حالا برای اینکه تبدیل معکوس به درستی گرفته شود، باید با ifftshift حالت اولیه برگردانده شود.

```
img_back = np.fft.ifft2(f_ishift)
```

استفاده از تبدیل فوریه معکوس دوبعدی (ifft2) برای تبدیل تصویر از حوزه فرکانس به حوزه مکان. خروجی این مرحله یک عدد مختلط (Complex) است.

```
img_back = np.abs(img_back)
```

به دلیل ماهیت تبدیل فوریه، خروجی img_back دارای بخش حقیقی و موهومی است. ما برای نمایش تصویر، اندازه (Magnitude) آن را با abs محاسبه می‌کنیم.

```
img_back = cv2.normalize(img_back, None, 0, 255, cv2.NORM_MINMAX)
```

مقادیر پیکسل‌های تصویر ممکن است خارج از محدوده استاندارد (۰ تا ۲۵۵) باشند. این دستور مقادیر را به محدوده [۰, ۲۵۵] نگاشت می‌کند تا تصویر به درستی نمایش داده شود.

```
img_back = np.uint8(img_back)
```

تبدیل data type به uint8 (عدد صحیح ۸ بیتی بدون علامت) که فرمت استاندارد تصاویر سیاه و سفید در کتابخانه OpenCV است.

در ادامه نیز، با کتابخانه matplotlib نتیجه نمایش و ذخیره میشود.

Function for testing filters

```
def test_filters()
```

تابع (test_filters) در واقع یک روش ارزیابی سیستماتیک است که عملکرد فیلترهای مختلف را با تغییر پارامترهای کلیدی ارزیابی می‌کند. این تابع با استفاده از حلقه‌های تو در تو، فیلترهای پایین‌گذر ایده‌آل، باترورث و بالاگذر ایده‌آل را با مقادیر مختلف شعاع پرش و مرتبه فیلتر تولید می‌کند. برای هر ترکیب، تابع apply_filter که قبلاً تعریف شده فراخوانی می‌شود تا فیلتر اعمال شده و نتایج تصویری به طور خودکار ذخیره شوند.

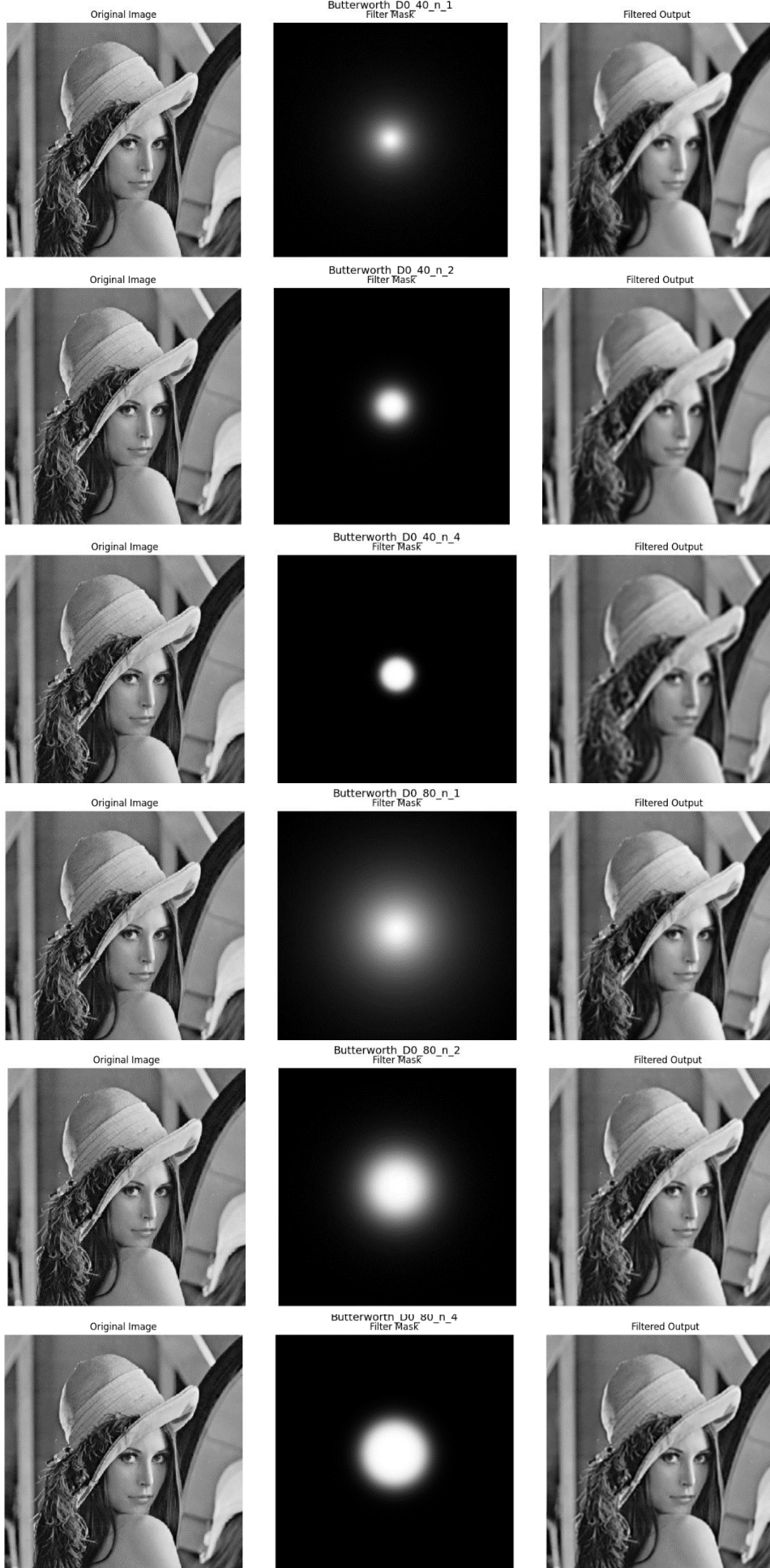
علاوه بر نمایش بصری، این تابع یک لیست به نام comparison_data تشکیل می‌دهد که به عنوان یک گزارش ساختاریافته عمل می‌کند. در این لیست، مشخصات هر فیلتر (نوع فیلتر، پارامترها و ویژگی‌های رفتاری مثل میزان نویز رینگینگ یا تیزی لبه‌ها) ذخیره می‌شود تا در نهایت امکان مقایسه دقیق تأثیر ریاضی این فیلترها بر روی تصویر فراهم گردد. در واقع این کد، فرآیند تست فیلترها را به یک آزمایش علمی تکرارپذیر و خودکار تبدیل کرده است.

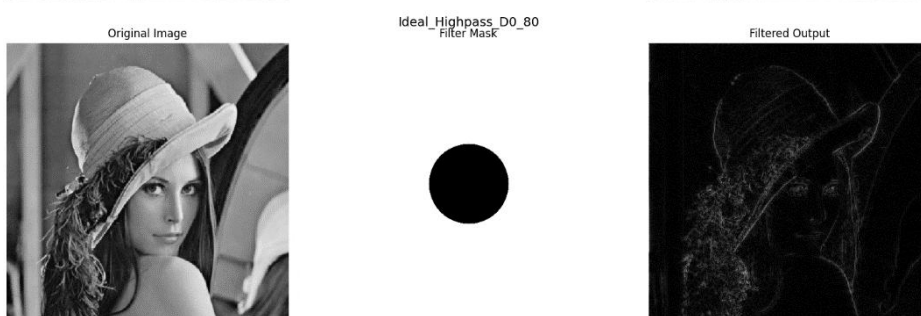
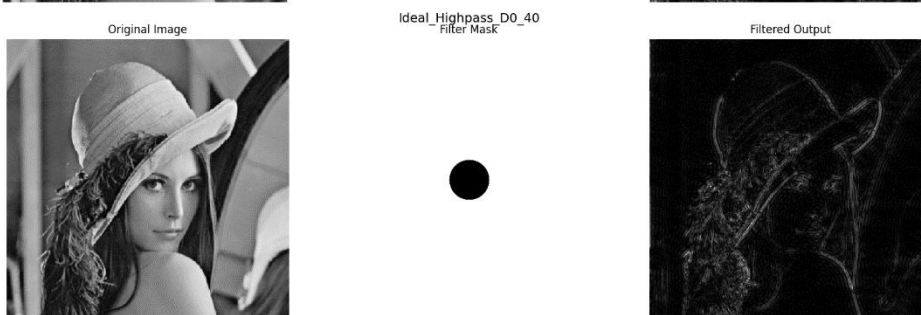
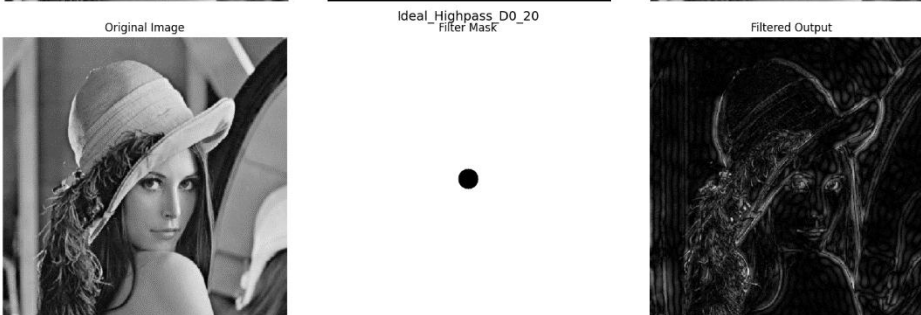
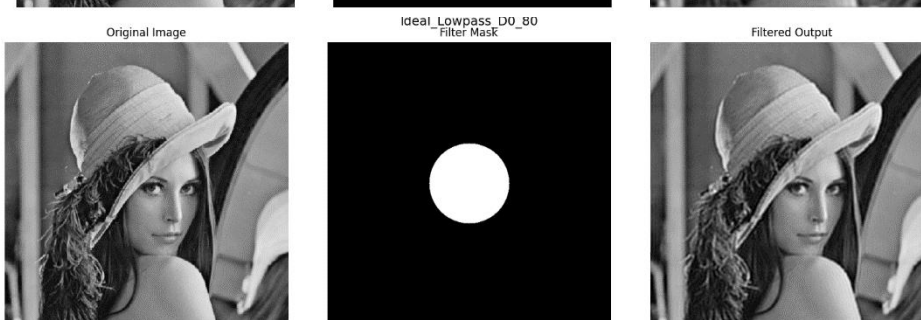
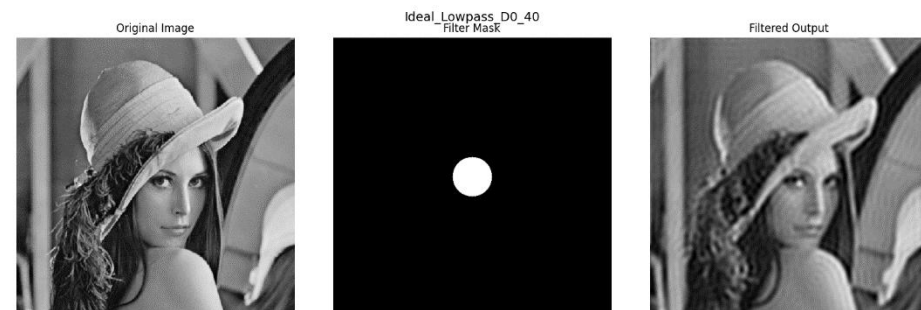
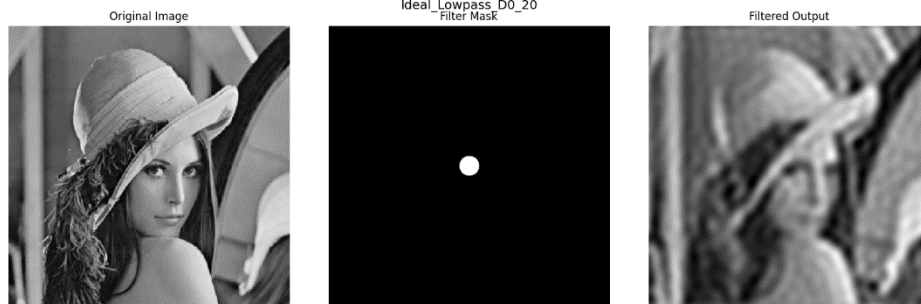
RUN ALL EXPERIMENTS

Test filters

تابع ارزیابی فیلتر فراخوانی شده و نتایج نمایش داده میشوند و ذخیره میشوند.







Create comparison table

در این بخش، داده‌های لیست comparison_data به یک DataFrame تبدیل در نهایت، با استفاده از متد to_csv این اطلاعات در یک فایل متنی با فرمت CSV ذخیره می‌گردد.

Filter Comparison Table

Filter Type	D0	Order n	Cutoff Behavior	Visual Effect
Ideal Low Pass	20	-	Sharp Cutoff	Strong Ringing
Ideal Low Pass	40	-	Sharp Cutoff	Strong Ringing
Ideal Low Pass	80	-	Sharp Cutoff	Strong Ringing
Butterworth	20	1	Smooth Cutoff	Moderate Ringing
Butterworth	20	2	Smooth Cutoff	Moderate Ringing
Butterworth	20	4	Smooth Cutoff	Moderate Ringing
Butterworth	40	1	Smooth Cutoff	Moderate Ringing
Butterworth	40	2	Smooth Cutoff	Moderate Ringing
Butterworth	40	4	Smooth Cutoff	Moderate Ringing
Butterworth	80	1	Smooth Cutoff	Moderate Ringing
Butterworth	80	2	Smooth Cutoff	Moderate Ringing
Butterworth	80	4	Smooth Cutoff	Moderate Ringing
Ideal High Pass	20	-	Sharp Cutoff	Strong Edge Enhancement
Ideal High Pass	40	-	Sharp Cutoff	Strong Edge Enhancement
Ideal High Pass	80	-	Sharp Cutoff	Strong Edge Enhancement

نتایج:

پدیده ringing:

Ringing نوعی نوسان یا هاله اطراف لبه‌ها در تصویر است. این پدیده به دلیل برش ناگهانی فرکانس‌ها ایجاد می‌شود. وقتی تابع فیلتر در حوزه فرکانس دارای تغییرات تند باشد، در حوزه مکان پاسخ سینکی (Sinc) ایجاد می‌کند که باعث نوسان می‌شود. بیشترین Ringing در فیلتر ایده‌آل مشاهده می‌شود، چون مرز برش آن ناگهانی است.

افزایش مرتبه n باعث تیزتر شدن مرز فیلتر، نزدیک شدن به فیلتر ایده‌آل و افزایش Ringing می‌شود.

فیلتر ایده‌آل دارای برش بسیار Sharp است که به دلیل رفتار پله‌ای، منجر به ایجاد نویزهای لبه‌ای (Ringing Artifacts) در تصویر می‌شود. در مقابل، فیلتر Butterworth با استفاده از یک تابع نرم (Smooth)، گذار ملایم‌تری بین فرکانس‌های عبوری و حذفی ایجاد می‌کند که باعث کاهش چشمگیر نوسانات ناخواسته و بهبود کیفیت بصری تصویر می‌گردد.

نتیجه گیری:

- فیلترهای پایین‌گذر باعث حذف جزئیات و هموارسازی تصویر می‌شوند.
- فیلترهای بالاگذر موجب تقویت لبه‌ها و جزئیات می‌شوند.
- فیلتر Butterworth نسبت به فیلتر ایده‌آل کیفیت بصری بهتری دارد.
- پدیده Ringing ناشی از برش ناگهانی در حوزه فرکانس است.